

Vision-Based Surface Classification for Friction-Aware Autonomous Racing

Ethan Sanchez
University of Pennsylvania
Philadelphia, PA
etsa@seas.upenn.edu

Avani Narula
University of Pennsylvania
Philadelphia, PA
avnarula@seas.upenn.edu

Yuntong Li
University of Pennsylvania
Philadelphia, PA
li51@seas.upenn.edu

Nandagopal Vidhu
University of Pennsylvania
Philadelphia, PA
nvidhu@seas.upenn.edu

Abstract—Autonomous vehicles operating in real-world environments must continuously adapt to changing terrain conditions that alter tire-road friction and directly affect handling performance. Traditional autonomous racing controllers typically assume a constant friction model, which can lead to degraded path tracking and unsafe maneuvers when transitioning between surfaces with different traction properties. In this project, we developed a vision-based surface classification pipeline for the RoboRacer platform that uses a forward-facing Intel RealSense camera to classify upcoming terrain and adjust controller behavior accordingly. A convolution neural network (CNN) was trained to identify road surface classes which were mapped to corresponding friction levels. These friction estimates were then integrated into a modified pure pursuit controller that dynamically adjusted parameters based on predicted surface conditions. Experimental evaluation on a multi-surface track demonstrated improvements in path tracking stability and reduced controller aggressiveness on low-friction surfaces compared to a fixed-parameter baseline controller. Our results suggest that incorporating predictive terrain awareness into lightweight racing controllers can improve both safety and consistency in autonomous driving systems.

Demo video: <https://youtu.be/PTKwRxNZAGY>

I. INTRODUCTION

Autonomous racing presents a challenging control problem in which vehicles must balance aggressive trajectory tracking with dynamic stability at high speeds. Many classical racing controllers, such as pure pursuit [1], [2] and model predictive control, perform well in structured environments but typically assume uniform road surfaces and constant tire-road interaction. In practice, however, real-world environments contain surfaces with significantly different friction properties, directly affecting acceleration, braking, and cornering behavior [3].

For small-scale autonomous racing platforms such as RoboRacer [4], these changes in traction can greatly impact controller performance due to the lightweight chassis and sensitivity to wheel slip. To address this limitation, our project investigates whether visual perception can be used to predict upcoming surface conditions and proactively adapt controller behavior before the vehicle experiences a loss of traction.

Our system combines a forward-facing camera with a lightweight vision model to classify terrain ahead of the vehicle in real time. These predictions are then used to dynamically modify parameters in a friction-aware pure pursuit controller, allowing the vehicle to adapt its driving behavior across different surface conditions.

II. PROBLEM MOTIVATION

Most autonomous racing pipelines focus heavily on localization, planning, and control while assuming environmental conditions remain constant throughout operation. In practice, vehicles often encounter abrupt transitions between surfaces with different traction characteristics, leading to overshoot, unstable cornering, and inconsistent performance.

We observed this issue directly by running a standard pure pursuit controller on an ice rink using parameters originally tuned for high-traction surfaces. The vehicle frequently slipped during turns, lost localization accuracy, and either turned too early or failed to maintain enough traction to stay within the track boundaries. These results demonstrated that a single fixed controller is insufficient for mixed-surface operation and that adaptive controller switching is necessary.

Existing friction estimation methods typically rely on wheel slip, inertial measurements, or observed vehicle dynamics [3]. While effective, these approaches are reactive because the vehicle must first experience degraded traction before adaptation can occur. More recently, learning-based controllers have been proposed that adapt online to multi-friction surfaces, for example using ensembles of Gaussian processes to model surface-dependent residual dynamics [5]; these too operate on observed dynamics rather than upcoming terrain. Our motivation was to develop a predictive friction-awareness system that allows the controller to adapt before entering a new surface region. By using vision to classify terrain ahead of the vehicle, the system can estimate expected friction conditions and proactively adjust control behavior in real time.

Although this work is motivated by autonomous racing, the same challenges exist in commercial autonomous driving. Road conditions such as snow, ice, gravel, or wet pavement significantly affect available tire traction and require different driving behavior to maintain vehicle stability. A predictive vision-based controller adaptation system could improve the safety and reliability of autonomous vehicles operating in dynamic real-world environments.

III. TECHNICAL APPROACH

The solution pipeline began with a forward-facing Intel RealSense camera capturing images of the surface directly ahead of the vehicle. These images were passed into a pretrained vision classifier to estimate the surface friction level. Based

on the predicted friction conditions, the pure pursuit controller parameters were continuously adjusted in real time to adapt the vehicle’s driving behavior.

A. Pure Pursuit Baseline

As a baseline, we implemented a standard pure pursuit controller [1] for waypoint tracking on the RoboRacer platform. The pure pursuit algorithm computes steering commands by selecting a target point at a specified lookahead distance and calculating the circular arc required to intersect it.

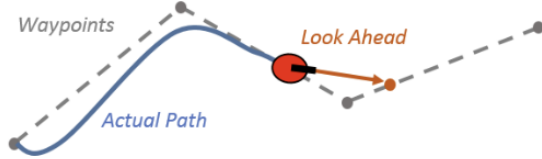


Fig. 1. Pure pursuit algorithm visualization.

For our specific implementation, we utilized a set of fixed waypoints to generate a smooth, continuous spline around the track, subsequently interpolating evenly spaced target points along this path. By evaluating the curvature of the raceline at each point, the system generated optimized target speeds and lookahead distances. To both ensure vehicle stability and pursue an aggressive trajectory we adjusted these target speeds using maximum lateral and longitudinal acceleration parameters, which dictated the vehicle’s cornering aggressiveness and straightaway acceleration. Crucially, this baseline controller operated using a single set of constant, uniformly tuned parameters across all surface types. This provided a reliable control group to evaluate whether our vision-based, friction-aware tuning meaningfully improved robustness and tracking accuracy on mixed-surface tracks.

B. Vision Model

The vision model is a fine-tuned MobileNetV3-Small convolutional network [6], chosen for its favourable accuracy-to-latency trade-off on the embedded NVIDIA Jetson platform aboard the RoboRacer car. The backbone is initialised from ImageNet-1K pretrained weights [7] and the final 1000-way classifier head is replaced with a four-way softmax over the surface classes used in our experiments (carpet, cloth, sticky sheet, and tile).

Training data was collected directly from the on-vehicle forward-facing Intel RealSense camera using a dedicated recorder ROS node that crops a fixed bottom-centre region of interest (ROI) of 200×160 pixels from the 640×480 RGB frame. With the camera mounted approximately 20 cm above the ground, this ROI corresponds to a patch of terrain roughly 30–50 cm in front of the bumper – close enough to be a reliable predictor of the surface the wheels are about to encounter, but far enough to give the controller time to react. We collected 801 hand-labelled patches across the four classes, then split them into training and validation manifests for fine-tuning. At

training time, patches are resized to 224×224 and normalised with the standard ImageNet channel statistics.

For deployment, the fine-tuned PyTorch checkpoint is exported to ONNX and then built into an FP16 TensorRT [8] engine on the Jetson. At inference time the classifier ROS node grabs frames from `/camera/rgb/raw`, applies the same fixed crop and normalisation, runs the engine, and emits the top-1 material label at a capped 10 Hz. Because frame-by-frame predictions can flicker at surface boundaries, the raw labels are passed through a five-frame majority-vote ring buffer; only the smoothed label is published.

The smoothed material label is then mapped to a discrete friction tier (low, medium, or high) via an editable YAML lookup file and published on `/surface/friction`. We deliberately chose discrete tiers rather than a continuous coefficient regression: a small number of tiers makes it tractable to hand-tune one full Pure Pursuit profile per tier and gives stable, auditable transitions between profiles. Continuous friction-coefficient estimation is left as future work.

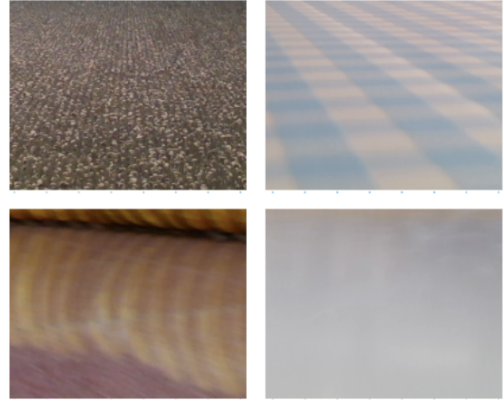


Fig. 2. Training images collected from the on-vehicle RealSense camera for the multi-surface track. Each row shows examples of the fixed bottom-centre ROI patch fed to the classifier, for one of the four surface classes (carpet, cloth, sticky sheet, tile).

C. Vision Aware Pure Pursuit

We extended our baseline controller by incorporating surface predictions from the vision model into the control loop. We first saved parameter profiles for each surface type, which were hand tuned to operate safely and aggressively. At each timestep, the predicted surface class was used to modify controller parameters. With the most important parameters being max speed, min speed, lookahead gain, and max longitudinal and lateral acceleration.

On higher-friction surfaces, the controller increased these parameters to enable more aggressive trajectory tracking. On lower-friction surfaces, these parameters were reduced to limit wheel slip and improve stability during turning and braking maneuvers. This design allowed the controller to proactively adapt behavior before encountering a surface transition, rather than reacting only after slip or instability had already occurred.

IV. IMPLEMENTATION

A. Software Architecture

The system was implemented using the Robot Operating System (ROS 2 Humble) [9], which allowed the perception, mapping, and control components to communicate through a modular node-based architecture. First, the environment was mapped using SLAM and particle filter localisation [10] so the vehicle could estimate its position on the track in real time. A waypoint generation node was then used to manually select and save waypoints along the desired racing line. These waypoints were stored in a CSV file and accessed by the pure pursuit controller during autonomous operation.

The vision classifier and adaptive pure pursuit controller were implemented as separate ROS nodes. The forward-facing Intel RealSense camera continuously captured images of the terrain ahead of the vehicle, and the classifier node processed these images using the pretrained model to predict the current surface type. The predicted terrain label was then published to a ROS topic in real time.

The adaptive pure pursuit node subscribed to this topic while simultaneously tracking the waypoint path. Based on the detected surface type, the controller loaded parameter sets from a YAML configuration file containing predefined surface classifications. Each parameter set represented a controller configuration specifically tuned for a particular terrain condition, allowing the vehicle to adapt its behavior as surface friction changed.

A visualization interface was created in Foxglove to display the waypoint spline and current lookahead point using topics published by a debug node. The Foxglove setup also included a panel showing the surface type currently detected by the vision classifier in real time.

B. Simulation Setup

The controller was first tested in simulation, where the surface friction condition was represented as a ROS parameter that could be manually changed during runtime. This allowed us to observe how the vehicle adapted its steering behavior and speed in response to the parameter sets associated with each surface type.

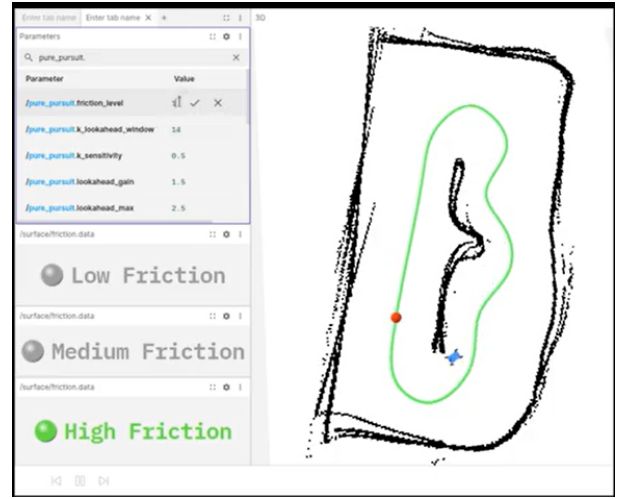


Fig. 3. Simulation Foxglove visualizer.

C. Hardware Setup

The software architecture was deployed on the RoboRacer platform using its onboard NVIDIA Jetson computer. An Intel RealSense camera was mounted at the front of the vehicle, as shown in the figure.



Fig. 4. RoboRacer car with RealSense camera.

V. TESTING

Experiments were conducted using the RoboRacer platform on two different tracks. Each experiment evaluated both the baseline pure pursuit controller and the vision-aware friction-adaptive controller under identical track and environmental conditions.

A. Ice and Carpet

The first experiment was designed to evaluate the system in a simple but highly visual mixed-surface environment. A test track was created in which half of the track consisted of very low-friction ice and the other half consisted of high-friction carpet, as shown below.



Fig. 5. Ice and carpet race track.

The experiment began by collecting training data for both surfaces. Images were gathered by manually driving the vehicle across the carpet and ice rink while recording camera data. Before integrating the vision classifier, the pure pursuit controller was independently tuned for each surface by observing vehicle behavior and adjusting the saved controller parameters for ice and carpet conditions separately.

TABLE I
TUNED PURE PURSUIT PARAMETERS FOR ICE AND CARPET SURFACES

Parameter	Ice	Carpet
Max Velocity	1.8	4.0
Min Velocity	0.5	1.5
Max Lateral Acceleration	1.5	4.0
Max Longitudinal Acceleration	1.0	2.2
Turning Gain	1.0	1.5
Lookahead Gain	1.5	1.5
Lookahead Min	0.7	0.7
Lookahead Max	2.5	2.5
Lookahead Window	14	18

After training the vision classifier, the vision-aware pure pursuit system was deployed on the mixed-surface track. The controller successfully tracked the waypoints by slowing the vehicle on ice while increasing speed on carpet surfaces. When the high-friction carpet controller was used alone on the ice surface, the vehicle frequently lost traction and spun out. In contrast, the adaptive controller was able to take advantage of the higher speeds possible on carpet while maintaining stability on the ice, demonstrating the benefit of predictive friction-aware controller switching.

B. Tile, Carpet, Cloth, and Sticky Sheet

Once the controller demonstrated reliable performance on the simple binary ice-versus-carpet experiment, we expanded testing to a more complex mixed-surface environment. The second experiment evaluated the system on a track containing multiple terrain types, including plastic cloth, tile, carpet, and sticky sheets. These surfaces introduced a wider range of traction conditions and more frequent transitions, creating a more realistic and challenging driving scenario for the adaptive controller.

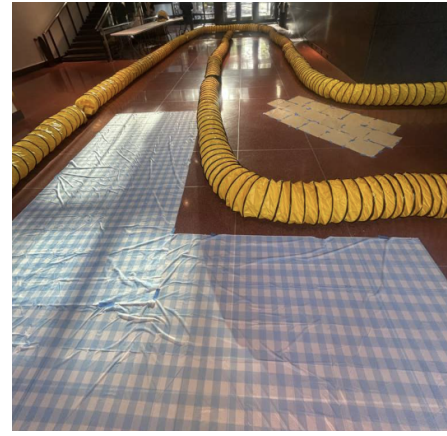


Fig. 6. Multi surface track with tile, carpet, plastic table cloth, and sticky sheets.

Training data was collected for each surface type by driving the vehicle across the track while recording images from the forward-facing camera. Individual pure pursuit parameter sets were then manually tuned for each terrain condition to account for differences in available traction and turning behavior. After training the classifier, the adaptive vision-aware pure pursuit system was deployed on the full mixed-surface track.

For each controller configuration, lap time and qualitative vehicle stability were recorded and compared. Additional observations included controller smoothness during surface transitions, frequency of overshoot events, consistency of trajectory tracking, and the vehicle's ability to recover from traction loss. Particular attention was given to how effectively the controller adjusted speed and steering behavior before entering new terrain regions.

RESULTS

The most important result of our experiments was qualitative rather than quantitative: on the mixed ice-and-carpet track, the aggressively tuned baseline pure pursuit controller – the controller that produced the fastest lap on pure carpet – was *unable to complete a lap at all*. The car carried carpet-tuned speeds into the ice section, lost traction, spun out, and either left the track or stalled against the rink wall. In other words, the headline baseline does not have a lap time on the mixed track to report; it failed to finish. The conservatively (ice-tuned) baseline did complete laps on the mixed track, but at the cost of driving the entire lap, including the high-traction carpet sections, at ice-safe speeds. The vision-aware adaptive controller was the only configuration that both completed laps and exploited the carpet sections, by swapping to the high-friction profile before the carpet patch and back to the low-friction profile before the ice. Table II summarises these three outcomes.

For reference, when each controller is run on its own home surface in isolation, the carpet-tuned controller laps the carpet-only track in roughly 7.8 s and the ice-tuned controller laps the ice-only track in roughly 13.0 s. The adaptive controller's 10.8 s mixed-track lap sits between these two single-surface

TABLE II
CONTROLLER OUTCOMES ON THE MIXED ICE-AND-CARPET TRACK.

Controller	Mixed-track outcome	Lap time
Aggressive baseline (carpet-tuned PP)	Spun out on ice; did not complete a lap	—
Conservative baseline (ice-tuned PP)	Completed; slow on carpet	13.0 s
Vision-aware adaptive PP	Completed; fast on carpet, stable on ice	10.8 s

times – which is precisely what we want, since the lap is half ice and half carpet. The headline finding is not the 10.8 s number itself, but the contrast between rows 1 and 3 of Table II: the friction-aware controller is the only one of the three that combines the aggressive baseline’s high carpet speed with the conservative baseline’s ability to keep the car on the track on ice.

Beyond raw outcomes, the adaptive controller also demonstrated smooth transitions between terrain regions and reduced spin-out events compared to a fixed controller configuration. The system was able to slow down before entering low-friction surfaces and accelerate again on high-traction terrain, resulting in more stable cornering and improved trajectory tracking throughout the course.

The vision classifier was able to reliably distinguish between highly different surface types such as ice, carpet, and tile under consistent lighting conditions. Misclassifications occurred most often near surface boundaries or when reflections were present on the ice surface; the five-frame majority vote absorbed most of this flicker before it reached the controller.

VI. CONCLUSION

This project demonstrated that vision-based terrain classification can be successfully integrated with a pure pursuit controller to create a friction-aware autonomous racing system. The adaptive controller was able to modify vehicle behavior in real time based on upcoming surface conditions, improving stability and maintaining higher overall performance across mixed-terrain tracks.

We concluded that the surface classifier performed reliably for a small set of pretrained surfaces and that longitudinal and lateral acceleration limits had the greatest impact on controller tuning across different friction conditions.

A. Challenges and Solutions

A few challenges shaped the final design of the system. First, the surface classifier struggled to generalise across visually similar but materially distinct surfaces – a model trained on a single carpet shade misclassified an unseen carpet, since the network had latched onto incidental colour and texture cues rather than the underlying material. We mitigated this by widening the per-class training set across lighting and material variants, so each label covers a broader visual distribution. Second, when the cropped ground ROI straddled two materials at a transition, raw single-frame predictions oscillated between classes; without smoothing this would have

caused the controller to flap between profiles mid-corner. We addressed this with a five-frame majority vote inside the classifier node, so a friction-tier change has to be supported by several consecutive frames before it propagates to the controller. Third, each surface required its own carefully hand-tuned pure pursuit profile, and this tuning had to be done on the actual surface to find the edge of stable behaviour – a tedious but unavoidable cost. We accepted the cost for this project, noting that this workflow mirrors the way production automotive controllers are tuned and validated extensively across conditions; eliminating the manual loop via continuous friction-coefficient regression is left to future work. Finally, the current pipeline always commits to its top-1 prediction; for a real-world deployment, both substantially more training data and an explicit low-confidence handling path (for example, falling back to the most conservative friction tier when softmax confidence drops below a threshold) would be needed.

B. Limitations

One limitation of the current system is that controller parameters had to be manually tuned and pretrained for each individual surface type. In addition, the vision classifier was trained on a limited set of terrain conditions, reducing its ability to generalize to unseen surfaces or changing lighting conditions.

C. Future Work

Future work will focus on developing a more automated friction estimation pipeline capable of directly predicting surface friction coefficients from camera images rather than classifying discrete terrain categories. This could allow controller parameters to be adjusted continuously without requiring manual tuning for each surface.

Additional improvements include increasing the robustness and generalization capability of the vision classifier across a wider variety of terrains and environmental conditions. We also plan to investigate integrating obstacle detection and avoidance using camera-based perception to identify hazards such as potholes or debris.

Finally, future work could explore extending the system to model-based controllers such as Model Predictive Control (MPC), where friction estimates could dynamically modify cost function weights and vehicle constraints in real time.

ACKNOWLEDGMENT

We thank the instructors and TAs of the ESE 6150 RoboRacer course at the University of Pennsylvania for guidance and direction.

REFERENCES

- [1] R. C. Coulter, “Implementation of the pure pursuit path tracking algorithm,” Robotics Institute, Carnegie Mellon University, Pittsburgh, PA, Tech. Rep. CMU-RI-TR-92-01, 1992.
- [2] J. M. Snider, “Automatic steering methods for autonomous automobile path tracking,” Robotics Institute, Carnegie Mellon University, Pittsburgh, PA, Tech. Rep. CMU-RI-TR-09-08, 2009.
- [3] S. Khaleghian, A. Emami, and S. Taheri, “A technical survey on tire-road friction estimation,” *Friction*, vol. 5, no. 2, pp. 123–146, 2017.

- [4] M. O’Kelly, V. Sukhil, H. Abbas, J. Harkins, C. Kao, Y. V. Pant, R. Mangharam, D. Agarwal, M. Behl, P. Burgio, and M. Bertogna, “F1/10: An open-source autonomous cyber-physical platform for experimental research,” *arXiv preprint arXiv:1901.08567*, 2019.
- [5] T. Nagy, A. Amine, T. X. Nghiem, U. Rosolia, Z. Zang, and R. Mangharam, “Ensemble Gaussian processes for adaptive autonomous driving on multi-friction surfaces,” in *Proceedings of the 22nd World Congress of the International Federation of Automatic Control (IFAC)*, 2023.
- [6] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, “Searching for MobileNetV3,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1314–1324.
- [7] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.
- [8] NVIDIA Corporation, “NVIDIA TensorRT: High-performance deep learning inference library,” <https://developer.nvidia.com/tensorrt>, 2024, accessed 2026-05-12.
- [9] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [10] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.